TEAM 5 / GOLD MEMBER WEEK

EventHub

Maintenance of the Software Application

CEN 4020 AI Project — Part 6 | Call 6 | November 9, 2026

Jaideep Yarasingu — Gold member • John Bachta — teammate / reviewer

This is our final Team 5 Part 6 coursework report. I used the Call 6 incident to review our EventHub code, requirements, diagrams and tests. We kept John's working purchase implementation intact, added the maintenance features and regression coverage, and made one focused maintainability refactor after the final teammate/AI review. Open production decisions are recorded honestly rather than treated as approved.

What this submission covers

Deliverable	What I prepared	Weight
1. Changes after Call 6	New EARS requirements, updated C4 component view, currency and safety changes, Spanish localization.	20%
2. Maintenance process	Actual refactoring decisions, incident analysis, before/after design, AI-assisted review, errors found in tests, recovery and monitoring procedures.	60%
3. Updated code + tests	Modular Flask/SQLite application, regression tests, backup and monitor utilities, original Part 5 system suite, reproducible commands.	20%

Final Part 6 verification — completed November 9, 2026

The verified branch revision passed 160 unit/integration tests and 139 Selenium browser/load tests on my Mac (299 total, zero failures). All four GitHub Actions quality and system checks also passed on the earlier review revision. That evidence is preserved in report/last_run.json and report/test_dashboard.html. We then made one additional small formatting-helper refactor and added a new test in response to the final review. The new test passed in a local dependency-light run, but the full Flask/Selenium suite and GitHub Actions still need a fresh run on this exact updated revision; the 299 result is not presented as proof for code changed afterward.

The older results remain part of the development record: John’s Part 5 project reported 281 passing tests, while my first offline Part 6 check had 94 selected passes with dependency-related skips. Those earlier runs are superseded by the full 299-test Part 6 verification recorded on November 9. This proves the tested classroom build passed its configured checks; it does not establish a production deployment or resolve open operational decisions.

1. What changed after Call 6 (20%)

1.1 First, what the incident tells us

In the meeting, release 1.3 accidentally treated every currency as if it had two decimal places. That turned a ¥4,500 ticket into ¥45, and three students bought tickets for much less than intended. The team restored release 1.2 instead of debugging on the live site. The audit data helped explain what happened, but nobody was monitoring it to raise an early warning.

I checked our Part 5 code before making changes. Our `compute_quote()` already stores amounts in integer minor units and knows JPY has zero decimal places. So I did not claim to reproduce the fictional bug in our code. Instead, I left the price calculation alone, added explicit yen regression tests, and moved the new regional display code into a separate module that cannot write to orders.

1.2 Requirement and model updates

Call 6 request	New IDs	Implementation
Currency-safe display	pricing-018/019/020	Spanish and English display formatting isolated from the one price function; yen keeps zero decimal places.
Admin rate update test	admin-009	Next purchase uses new stored rate; previous purchase keeps its original rate.
Someone notices wrong totals	audit-009/010/011	Outlier scanner, persisted alert rows, admin acknowledgement, optional configured SMTP.
Spanish and regional dates	system-010/011	JSON translation catalog, language selector, localized event timestamps and currency presentation.
Backups and recovery	system-012/013	Consistent SQLite snapshot, SHA-256 integrity manifest, guarded restore operation.

The earlier IDs were kept. The additions bring the repo from 107 to 118 EARS requirements, and the original supersession links still pass the course checker. The Call 6 model adds Localization & Formatting, Order Anomaly Monitor, and Backup & Recovery components. `docs/part6-traceability.md` maps every change to the exact files and tests.

1.3 Customer decisions we did not invent

Call 6 gave us clear directions but did not name a human who receives anomaly alerts, approve a threshold, or specify a backup operator, schedule or retention policy. The meeting asked for a backup owner by Friday; no actual person was assigned in the materials we received. Q22-Q26 and RISK-006-010 remain open. The English interface still uses the legacy 'JPY 4,500' rather than the currency-native '¥4,500', and the en/es cookie switch is a language preference, not automatic regional detection. We state these as remaining work, not completed requirements.

2. The maintenance process (60%)

2.1 How I used the C4 diagram to guide changes

I described the new boundaries as small contracts: what a module receives, what it returns, what it is allowed to change, and what must remain true. That is more useful for maintaining the code than telling AI to “make the app cleaner.”

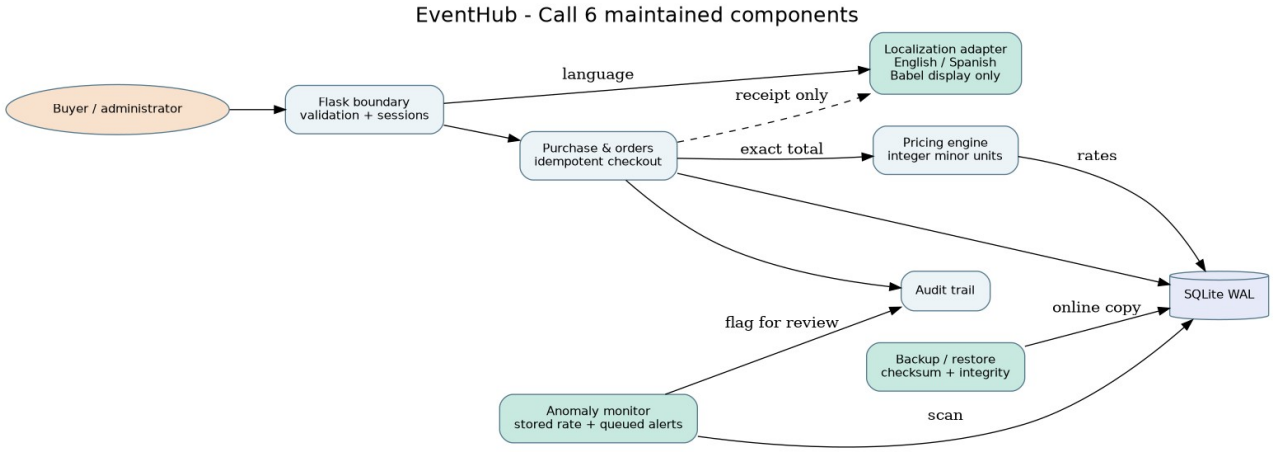


Figure 1. Updated component relationships. Green modules are new maintenance boundaries. The source architecture is in `workspace.dsl` and the drawing source is in `diagrams/Part6Components.dot`.

2.2 Changes that make future work easier

Before	After / reason
Pricing and display words were close to the same flow.	`compute_quote()` stays the only function that calculates the charged amount. `i18n.local_money()` is presentation-only and takes a finished integer amount.
English text lived in HTML templates.	English/Spanish JSON catalogs + a small adapter. A third language can be added without rewriting the order transaction.
Audit entries existed but no anomaly scanner did.	A separate scanner evaluates stored orders and queues one durable alert per order/reason. Notifications are outside the transaction.
WAL database had no project-level recovery tool.	SQLite online backup + integrity check, SHA-256 manifest and explicit offline restore confirmation.
web.py repeated English money and date selection, while i18n.py handled Spanish.	Both template helpers now delegate to i18n.py; old English amounts/dates and Spanish output are regression-tested. This is the focused, actual refactor.

2.3 One small refactor with a big safety benefit

The actual refactor chosen after team review was small and behavior-preserving. Earlier, web.py handled English money and date formatting inline and called i18n.py only for Spanish. That duplicated language decisions between two files. Now both template helpers call i18n.local_money() and i18n.local_date(), and the adapter owns the old English format plus the Babel-based Spanish format. Neither helper calculates or changes an amount charged.

```
# Money calculation (still the single source of truth)
quote = compute_quote(base_usd_minor, quantity, eligible, currency, rate)
order_total_minor = quote.total_minor   # saved in SQLite

# Call 6: display only, after the charge is settled
shown_to_user = local_money(order_total_minor, currency, language)
# JPY remains 4500 integer yen, never 45.00 after formatting.
```

This was a deliberate choice: I did not modify compute_quote() or the order transaction because they already protect integer minor-unit accounting and purchase atomicity. English still displays 'JPY 4,500' for old tests; Spanish retains regional formatting. A new test checks the English date string, English yen value, language fallback and Spanish output. Native yen symbols in English and reader-region detection remain separate, unapproved UI requirements rather than hidden changes.

2.4 How the AI-assisted changes were checked

For the main implementation, I used ChatGPT to explore module contracts and tests and compared the suggestions with Call 6. During the final teammate review, Claude's feedback specifically identified the long create_app() and place_order() functions and the duplicated language branches. I read those suggestions against the source and chose the low-risk formatting refactor, rather than blindly splitting the sensitive purchase and session code at the end of the project. The original Claude prompt was not preserved, so this report documents the received findings and our response, not a made-up prompt transcript.

Step	What I checked or changed
Baseline	Inspected Part 5 pricing, exchange-rate edits, SQLite WAL, audit writes, templates, existing tests and 281-test dashboard.
Generate focused work	Built separate localization, monitor and backup modules; added core Spanish strings and two additional Selenium scenarios.
Review an AI-written test	My initial parameterized yen test wrongly asserted `JPY 4,500` for USD, EUR and GBP rows. That created three real failures.
Correct the test, not the code	The price function was correct. I narrowed the yen-specific display assertion to the JPY case, reran tests, and kept the error in the evidence notes.
Regression & handoff	Added persisted-purchase tests for every currency and an administrator rate-edit regression. Ran EARS/C4 and traceability checks, then fixed Python 3.9 compatibility and browser-test isolation/timing problems and reran the final suite. The completed Mac run passed 299 tests.
AI-assisted simplification review	Claude feedback received through teammate review identified long web/purchase functions and repeated language decisions. I implemented the narrow locale adapter refactor, added a test and deferred risky transaction/session splitting.

The main thing I learned is that a generated test can be wrong even when the application is right. I should check an assertion against the requirement and the input values before changing production code just to make a test green.

2.5 Audit monitoring and the incident response

An audit record that nobody reads is only useful after someone reports an incident. The new `monitor.scan_orders()` checks historic orders using each order's saved USD unit price, exchange rate and discount. It does not use today's rates, because that would incorrectly flag purchases made before an administrator changed a rate.

If a saved order is far from the expected amount, the monitor writes one row to `maintenance_alerts`. The administrator page lists the alerts and allows acknowledgement without deleting the evidence. `scripts/check_order_anomalies.py` can send a message using a configured SMTP server; without a recipient, it keeps the alert visible and exits with a warning. We cannot say this pages an on-call human until somebody configures a real monitoring schedule and owns the inbox.

```
# Key property: the scanner is idempotent
new_alerts = scan_orders(connection, deviation_percent=25)  # provisional
# For a deliberately damaged JPY test order: first scan = 1, next scan = 0
# Acknowledgement records the administrator; it does not erase the alert.
```

2.6 Recovery and rollback are different problems

Call 6 says the team rolled the program back to release 1.2 in 11 minutes. That is a story in the assignment, not a rollback I performed on a live host. I wrote the release runbook instead: save a known-good code version, back up the database, stop checkout if pricing is wrong, restore the known-good code, then verify exact amounts before making a new patch.

For the database, simply copying `eventhub.sqlite3` while WAL mode is active can miss committed data. The new tool uses SQLite's own backup API and writes a checksum manifest. Restore is explicitly offline, requires confirmation, verifies the checksum and validates the resulting SQLite database. The owner, backup schedule, retention and offsite destination still need a decision.

2.7 Adding Spanish without another pricing bug

The selector lets users choose English or Spanish and stores that choice in a cookie. Translated labels and Spanish date/money presentation stay outside transaction logic. It does not detect a visitor's country and it does not convert venue-local times across time zones. English zero-decimal yen amounts are numerically correct but use the old JPY prefix rather than the requested native yen symbol. I left those differences visible in Q24 so the team can ask for customer approval before changing receipt and quote contracts.

2.8 Simplification review: what AI suggested and what we changed

The final teammate review included Claude suggestions focused on maintainability. It flagged create_app() in web.py (around 446 lines), place_order() in purchase.py (around 138 lines), and repeated en/es formatting decisions in the web layer. Those are real areas for improvement, but they carry different risks.

Implemented now: I moved the English-vs-Spanish money and date decision into i18n.py. web.py only supplies the active language. The new test checks the exact existing English date and JPY 4,500 output plus Spanish formatting and an unsupported-language fallback. This reduces coupling without changing any charge, database row, login/session behavior or CSRF handling.

Deferred with reasons: breaking create_app() into authentication, order and admin route modules would require a careful CSRF/session review. Breaking place_order() into helper functions would need to keep BEGIN IMMEDIATE as the one transaction owner, then rerun concurrent-purchase, duplicate-token and stock tests. Those are separate maintenance tickets, not edits to rush into this assignment.

This is also how I used AI responsibly: I treated the feedback as a shortlist, checked it against tests and component boundaries, and chose one change with a clear before/after and measurable regression condition. The detailed review is in docs/part6-ai-simplification-review.md.

3. Updated code and regression evidence (20%)

3.1 Tests that directly address the incident

Scenario	Expected result
$30 USD ticket	Charge exactly 3,000 cents and display USD 30.00 in English.
Same ticket converted at 0.92 EUR/USD	Charge exactly 2,760 EUR cents.
Same ticket converted at 0.78 GBP/USD	Charge exactly 2,340 GBP pence.
Same ticket converted at 150 JPY/USD	Charge exactly 4,500 yen, no two-decimal division.
Admin edits EUR 0.92 to 1.10	Next $18 ticket charges 1,980 EUR cents; older order stays 1,656.
Tampered JPY order of 27 versus expected 2,700	One durable anomaly alert; rerunning scan makes no duplicate alert.
Backup checksum changed after creation	Offline restore rejects the backup and preserves the destination.
Spanish display	Core UI translations and date/currency display change; original integer charge does not.

3.2 Exactly what I executed

Check	Result in this environment
Python compileall	PASS on application, test and maintenance scripts.
Course EARS/C4 gate	PASS: 118 requirements, 14 supersession link ends, zero failures. Structurizr CLI not available.
Requirement-to-test mapping	PASS: 111 active requirements, 108 with test tags and 3 documented exemptions (latest structural check).
Call 6 domain test file	PASS: 15 domain tests, including one added locale-adapter regression, in the limited local environment. Full Flask/Selenium retest of the new revision is pending.
Broader existing + new tests	Last full pre-refactor Mac run: 160 passed, zero failures. The new regression adds one test; no new full-suite count is claimed until rerun.
Previously supplied Part 5 result	281 passed (144 unit/integration, 137 browser/load) — archived Part 5 baseline only; not the Part 6 verdict.
New Selenium + full suite	Previously verified Mac Part 6 revision: 139 browser/load passed, 1 environment warning. With 160 unit/integration tests, 299 passed. Four GitHub Actions checks also passed on the previous PR commit. Fresh verification is required for the subsequent locale refactor.
New review revision status	Python compileall, course EARS/C4 and requirement traceability passed locally; 15 Call 6 domain tests passed. Flask/Selenium and Ruff are unavailable in this

	isolated environment, so a new complete Mac and GitHub check is still required.

The last completed pre-review full Mac run passed 160 unit/integration and 139 Selenium tests, 299 total; the related GitHub PR subsequently showed all four quality/system checks green. The last review introduced a small format-adapter refactor and one additional regression test (15 Call 6 domain tests passed in the limited local environment, up from 14). Because this container lacks Flask, Werkzeug, Selenium and Ruff, the updated full suite was not re-executed here. The final branch must be rerun before anyone records a new full-suite result. Old Part 5 and older failed trial artifacts have been separated from the submission evidence.

4. Final team submission and outstanding decisions

4.1 Included project files

File/folder	Why it matters
eventhub/i18n.py; locales/; templates/; static/	Core Spanish UI and display-only date/money formatting.
eventhub/monitor.py; scripts/check_order_anomalies.py	Alert logic, threshold, durable review queue and conditional SMTP.
eventhub/backups.py; scripts/backup_db.py; scripts/restore_db.py	Consistent WAL backups, checksums and an explicit restore guard.
features/*/requirements.md; workspace.dsl; diagrams/Part6Components.*	Changed requirements and revised model with old designs retained.
tests/test_call6_*; tests_system/test_call6_browser.py	New fast, HTTP and Selenium tests; check currency and admin workflows.
docs/release-and-recovery.md; docs/open-items.md; docs/risk-register.md	Rollback procedure, candid unresolved owner/alert and locale decisions, and operational risks.
docs/part6-ai-simplification-review.md; docs/part6-release-decisions.md	Actual AI-suggested simplification analysis, implemented/refused refactors, unresolved native yen/region and accountable operations owners.

4.2 Operational choices still awaiting customer assignment

- Q22: The 25% anomaly threshold is a demo assumption, not an approved production policy.
- Q23: An alert inbox/SMTP relay and a named on-call operator must be assigned; code alone will not notify anybody.
- Q24: Native-symbol English yen display (¥4,500), reader-region selection, translation approval and timezone policy are still pending; en/es is a manual cookie preference.
- Q25: Call 6 requested a backup owner by Friday, but no named person was supplied. Ownership, schedule, retention, offsite backup and drill still need assignment.
- Q26: Confirm the real deployment mechanism, restore a known-good release and document a timed rollback drill.

4.3 Final team statement

This is our final classroom maintenance report, not a production-release approval. The submission includes updated requirements, diagrams, implementation, an AI-assisted simplification review, regression tests and preserved verification evidence. We simplified the duplicated language logic without changing the purchase transaction. The next step for the team is to rerun full CI on this revised commit; the actual backup operator, alert recipient, region-specific formatting and live rollback drill still need decisions from the responsible people.